

UPDS JUDGE

Informe final de análisis, diseño y preparación para desarrollo

Proyecto: Plataforma web para concursos de programación competitiva.

Repositorio frontend: <https://github.com/Alex-Fernandez-2003/UPDS-JUDGE-FRONT.git>

Repositorio backend: independiente; URL pendiente de definición.

Backend: ASP.NET Core MVC / Web API.

Frontend documentado: React, Vite y TypeScript; su código no está presente en este checkout.

Integrantes

Wilson Yucra Rengifo
Cristhian Joel Amador Gallardo
Arnold Daniel Torrez Zarate
Daniel Javier Aramayo Mancilla
Enny Anaí Lopez Saldaña Beymar
Angelo Vasquez Acha
Alex Saul Fernandez Valdez

Julio de 2026

Índice

1. Introducción	2
2. Contexto estratégico y diagnóstico	2
2.1. Problema central	2
2.2. Causas y efectos	2
3. Solución y propuesta de valor	2
3.1. Visión	2
3.2. Objetivo general	3
4. Definición del MVP	3
5. Product Backlog	3
5.1. Historias críticas	4
6. Definition of Ready	4
7. Modelado UML	4
7.1. Modelo de contexto	4
7.2. Casos de uso	4
7.3. Clases de persistencia	4
7.4. Secuencia de envío	4
8. Arquitectura de software	5
8.1. Repositorio frontend	5
8.2. Estructura disponible en este checkout	5
8.3. Repositorio backend	5
9. Arquitectura de datos	5
9.1. Entidades	5
10.Reglas de ranking	8
11.Seguridad propuesta	8
12.Sprint 0 y fábrica de software	11
13.Ready to Sprint	11
14.Conclusiones	11
A. Fuentes internas disponibles	11
B. Archivos complementarios	11

1. Introducción

UPDS JUDGE es una propuesta de plataforma universitaria para gestionar concursos de programación competitiva. El sistema cubre registro, autenticación, roles, concursos, problemas, casos de prueba, envíos, evaluación automática, veredictos, ranking, congelamiento e historial.

La documentación se elaboró a partir de los recursos versionados: diagramas renderizados y sus fuentes PlantUML, capturas, el script SQL inicial y los documentos Markdown. Se distingue entre evidencia disponible y decisiones propuestas para no presentar como implementadas funciones que todavía están en planificación. En el árbol inspeccionado no existe la carpeta `frontend/`; por ello no se afirman dependencias, scripts, variables de entorno ni una ejecución local del cliente.

2. Contexto estratégico y diagnóstico

Cuando la organización de un concurso depende de herramientas separadas, los responsables deben registrar participantes, distribuir problemas, ejecutar soluciones y recalcular resultados de forma manual o fragmentada. Esto aumenta la carga operativa y reduce la trazabilidad.

2.1. Problema central

La comunidad académica no cuenta todavía con una plataforma integrada y trazable que permita organizar concursos de programación, evaluar automáticamente las soluciones y publicar resultados oportunos, lo que incrementa la carga operativa, el riesgo de inconsistencias y la dificultad para conservar historial y ranking confiables.

2.2. Causas y efectos

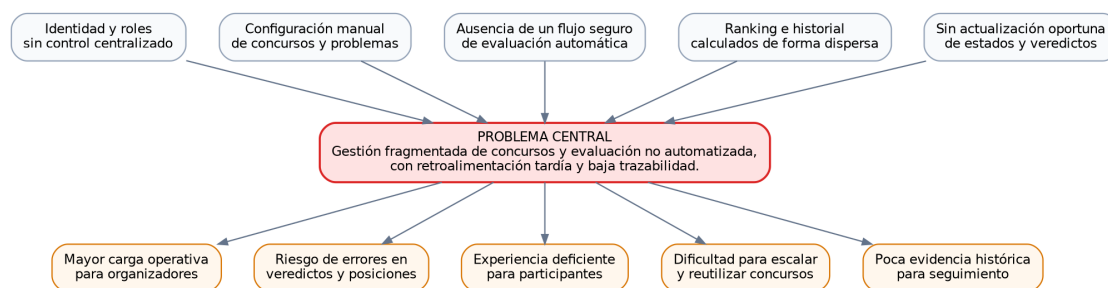


Figura 1: Árbol de problemas de UPDS JUDGE.

3. Solución y propuesta de valor

La solución propuesta es una aplicación web que unifica el ciclo completo de competencia. El frontend ofrecerá la experiencia de usuario; el backend aplicará seguridad y reglas; Judge0 se contempla para la ejecución aislada; PostgreSQL para la trazabilidad; y SignalR para la actualización de estados. Estas integraciones forman parte del alcance documentado y no se presentan como implementadas.

3.1. Visión

Para estudiantes y organizadores de concursos de programación, UPDS JUDGE es una plataforma que permite preparar competencias, evaluar soluciones automáticamente y consultar resultados trazables. A diferencia de un proceso manual, centraliza permisos, veredictos, ranking e historial.

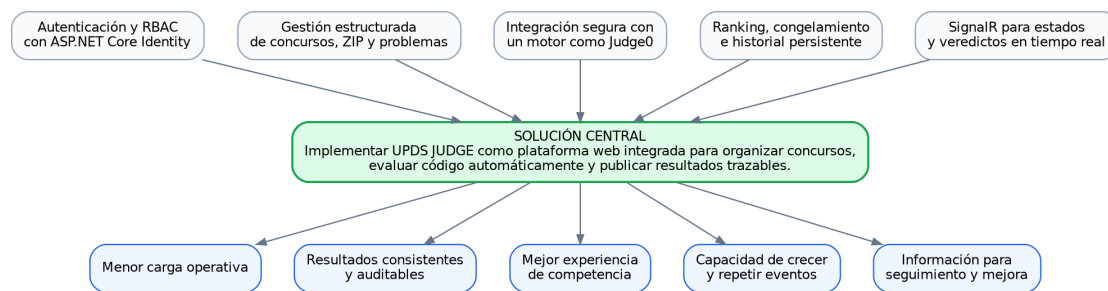


Figura 2: Árbol de soluciones propuesto.

3.2. Objetivo general

Diseñar e implementar un MVP de UPDS JUDGE que permita administrar concursos, recibir y evaluar soluciones de forma automatizada, comunicar veredictos en tiempo real y calcular rankings trazables, mediante un frontend independiente y un backend ASP.NET Core MVC/Web API separado.

4. Definición del MVP

El MVP incluye:

- Registro, inicio de sesión y autorización por roles.
- Creación de concursos, privacidad, duración, PDF y congelamiento.
- Importación de problemas y casos desde ZIP.
- Participación y consulta de problemas.
- Envío de C++, Python y C# a un motor de evaluación.
- Veredictos en tiempo real y recuperación por HTTP.
- Upsolving separado del ranking oficial.
- Ranking por resueltos y penalización.
- Historial personal filtrable.

El MVP no incluye detector de plagio, editor colaborativo, gestión universitaria general ni ejecución de código directamente dentro del proceso ASP.NET.

5. Product Backlog

La captura contiene 15 historias. La numeración salta de UJ-16 a UJ-18; no se inventó una UJ-17.

ID	Ép.	Historia	Prio.	Est.
UJ-05	E01	Como usuario nuevo, quiero registrarme con nombre, correo y contraseña, para tener acceso a la plataforma.	Alta	3 h
UJ-06	E01	Como usuario registrado, quiero iniciar sesión para obtener un token que identifique mis roles.	Alta	5 h
UJ-07	E01	Como Administrador de Roles, quiero asignar o revocar los roles Admin Concursos o Admin Roles a otros usuarios, para delegar la organización y administración de la plataforma.	Alta	3 h

ID	Ép.	Historia	Prio.	Est.
UJ-08	E02	Como Administrador de Concursos, quiero crear un concurso con nombre, descripción, fecha de inicio, duración, enlace al PDF y minutos de congelamiento, para publicar una competencia configurada.	Alta	5 h
UJ-09	E02	Como Administrador de Concursos, quiero subir un archivo ZIP con la estructura de carpetas A, B, C para que los casos de prueba se carguen automáticamente.	Crítica	1 d 5 h
UJ-10	E02	Como Administrador de Concursos, quiero asignar título, inciso (A, B), límite de tiempo en segundos y memoria en MB a cada problema extraído del ZIP.	Alta	3 h
UJ-11	E03	Como usuario, quiero ver una lista de concursos filtrados por En curso, Próximo y Finalizado.	Alta	3 h
UJ-12	E03	Como usuario, quiero ingresar la contraseña de un concurso privado para poder participar oficialmente.	Alta	5 h
UJ-13	E03	Como usuario dentro de un concurso, quiero ver la lista de incisos (A, B, C) y un botón para abrir el PDF del set de problemas alojado en Google Drive.	Alta	2 h
UJ-14	E04	Como usuario, quiero seleccionar un lenguaje (C++, Python o C#) y subir mi código para que sea evaluado.	Crítica	5 h
UJ-15	E04	Como usuario, quiero recibir el veredicto (Accepted, WA, TLE, MLE o CE) en tiempo real.	Crítica	1 d 5 h
UJ-16	E04	Como usuario, quiero enviar soluciones a concursos finalizados para practicar sin afectar el ranking original.	Media	2 h
UJ-18	E05	Como usuario, quiero ver el ranking calculado por problemas resueltos y penalización (minutos más 20 minutos por intento fallido).	Crítica	1 d
UJ-19	E05	Como Administrador de Concursos, quiero que el ranking deje de actualizarse públicamente cuando falten X minutos para el final.	Alta	3 h
UJ-20	E05	Como usuario, quiero ver todos mis envíos con filtros por concurso y resultado obtenido.	Alta	5 h

5.1. Historias críticas

Se seleccionan UJ-09, UJ-14, UJ-15, UJ-18 y UJ-19 por concentrar carga de archivos no confiables, evaluación asíncrona, tiempo real y reglas de ranking.

6. Definition of Ready

Una historia está lista cuando tiene rol, valor, criterios verificables, prioridad, estimación, diseño o referencia visual, contrato de API, reglas de autorización, datos persistentes, estados de error y dependencias identificadas.

Para las historias de evaluación se exige además un entorno Judge0 disponible, formato ZIP acordado y almacenamiento privado definido.

7. Modelado UML

7.1. Modelo de contexto

7.2. Casos de uso

7.3. Clases de persistencia

7.4. Secuencia de envío

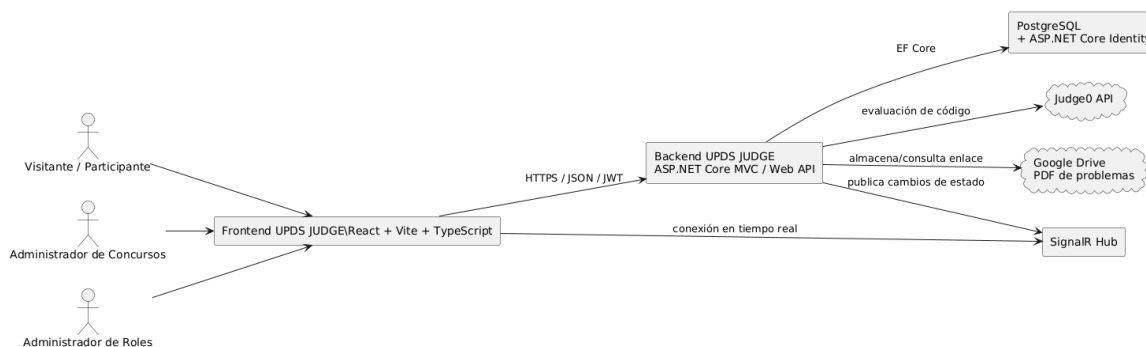


Figura 3: Contexto técnico y actores.

8. Arquitectura de software

8.1. Repositorio frontend

El frontend se referencia en <https://github.com/Alex-Fernandez-2003/UPDS-JUDGE-FRONT.git>. La decisión documentada es React con Vite y TypeScript en un repositorio independiente. Aunque los documentos de planificación sitúan el proyecto en `frontend/`, esa carpeta no existe en el árbol de este checkout; por ello no es posible verificar aquí sus dependencias, scripts, Tailwind CSS ni variables de entorno.

8.2. Estructura disponible en este checkout

El árbol inspeccionado contiene `database/script-inicial.sql`, `docs/` con documentos Markdown, fuentes PlantUML, imágenes, capturas, `informe-final.tex` y `informe-final.pdf`, además de los archivos raíz `README.md` y `.gitignore`. La carpeta `frontend/` no está presente. Por esa razón, la estructura documentada para el cliente debe entenderse como planificada y no como evidencia de una aplicación incluida en este checkout.

8.3. Repositorio backend

El backend se contempla en un repositorio independiente con ASP.NET Core MVC/Web API y URL pendiente de definición. Como la interfaz principal corresponde al frontend React independiente, el backend se orientará a exponer HTTP/JSON. Controllers, Models/DTO, Services, EF Core, SignalR e integración Judge0 son decisiones documentadas para su desarrollo; no existe evidencia local de su implementación ni de un contrato OpenAPI finalizado.

9. Arquitectura de datos

El modelo es preliminar y se conserva como referencia conceptual. La documentación propone delegar las credenciales de usuario a ASP.NET Core Identity y proteger los casos de prueba mediante almacenamiento privado; no hay migraciones ni implementación de backend disponibles en este repositorio para verificarlo.

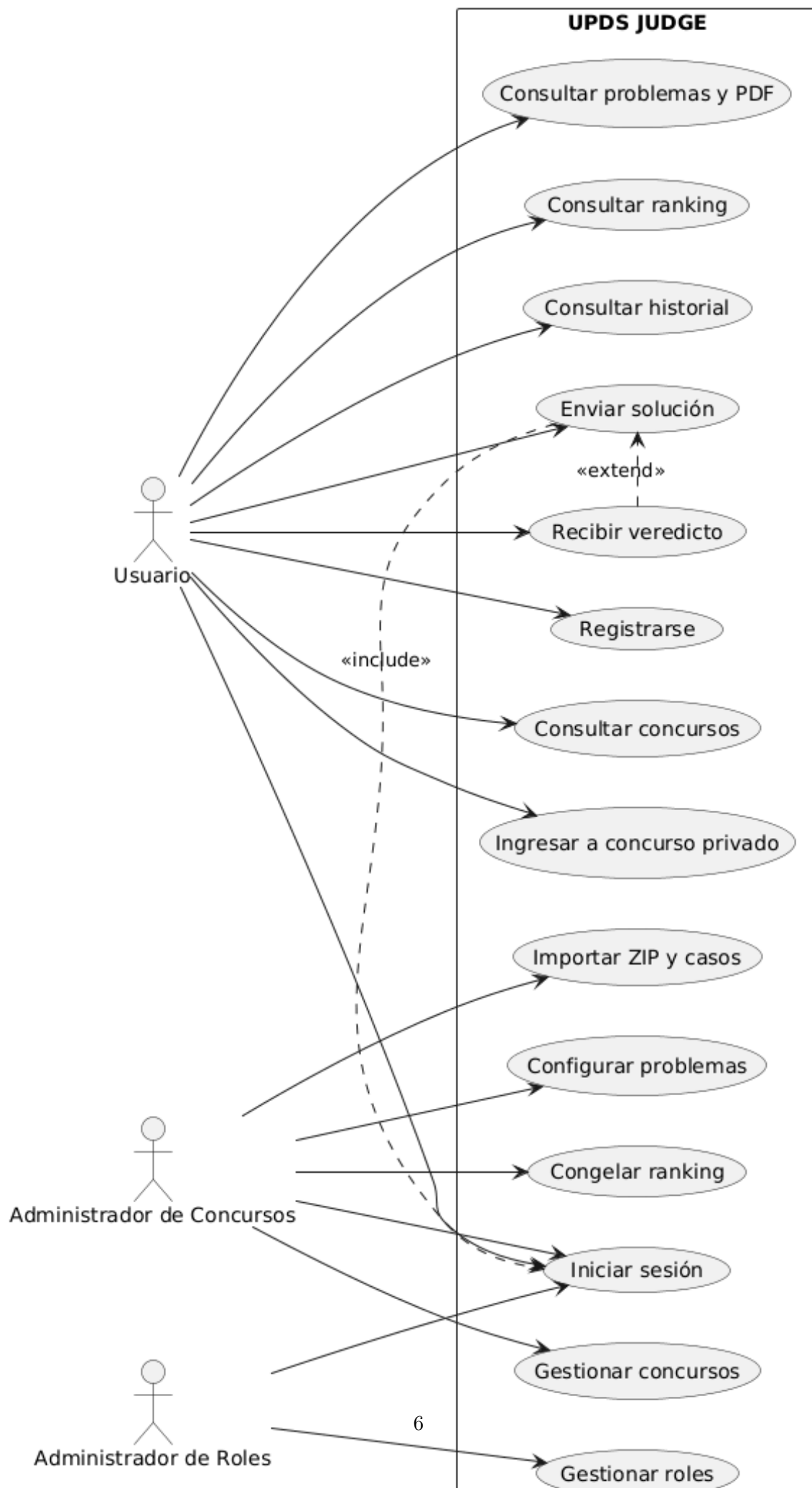
9.1. Entidades

Concursos: configuración, fechas, privacidad, PDF y congelamiento.

ParticipantesConcursos: inscripción única del usuario.

Problemas: inciso, título y límites.

CasosPrueba: referencias privadas a entradas y salidas esperadas.



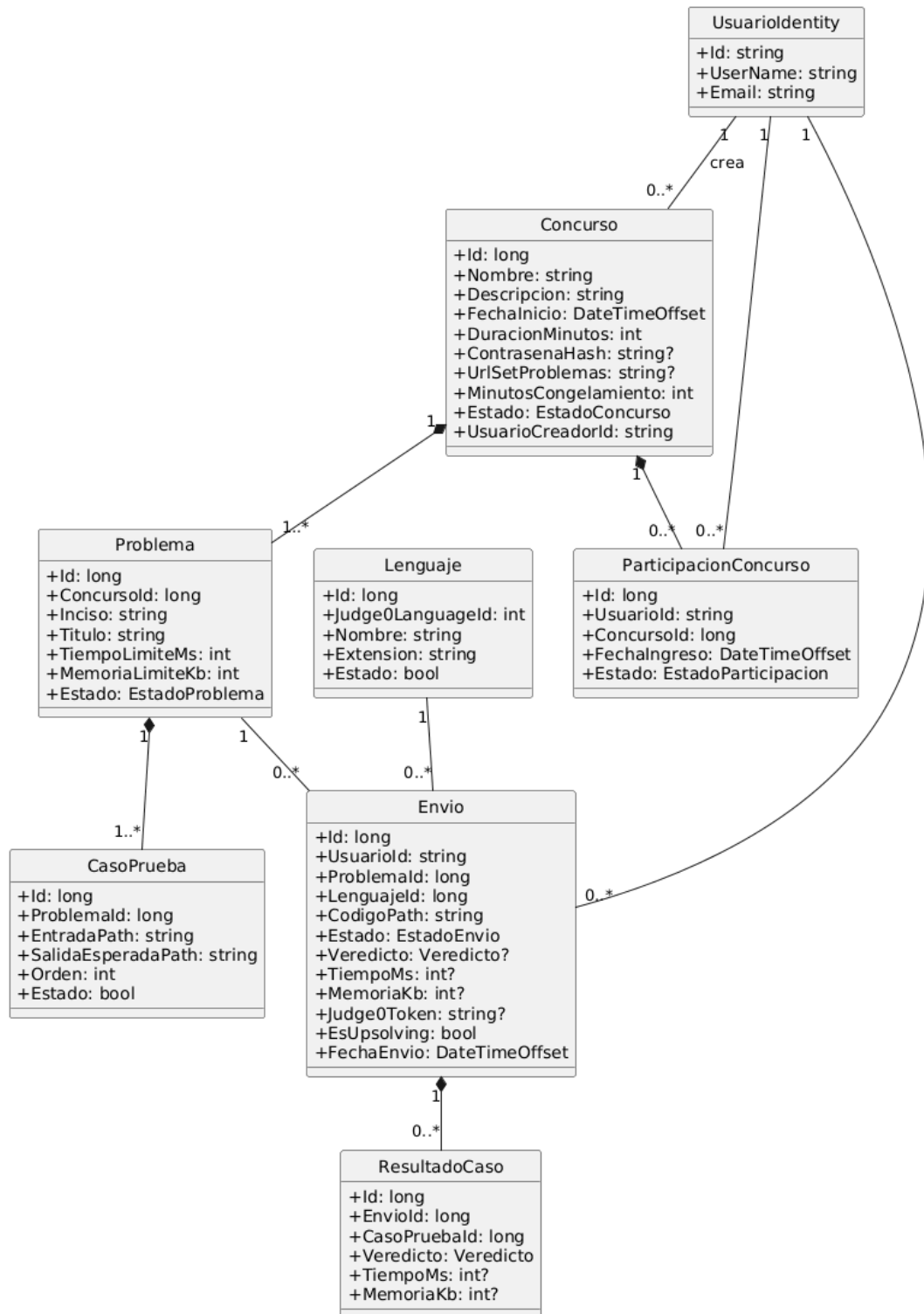


Figura 5: Clases de dominio y relaciones principales.

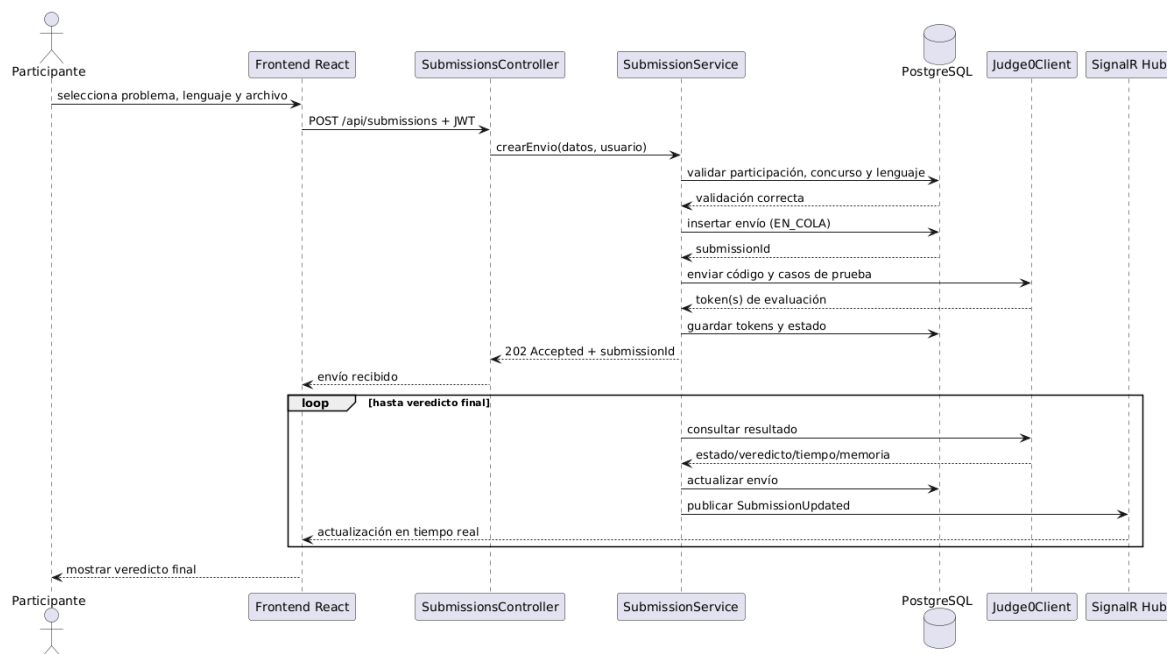


Figura 6: Flujo propuesto de envío y veredicto.

Lenguajes: mapeo con identificadores Judge0.

Envíos: trazabilidad, estado, veredicto, tiempo, memoria y upsolving.

ResultadosCasos: diagnóstico interno por caso.

10. Reglas de ranking

1. Ordenar por cantidad de problemas aceptados, descendente.
2. Para cada problema aceptado, sumar el minuto del primer Accepted y 20 minutos por intento fallido oficial anterior.
3. No penalizar problemas nunca aceptados.
4. Excluir todos los envíos de upsolving.
5. Durante el congelamiento, conservar un ranking público y otro real administrativo.

11. Seguridad propuesta

- Se propone Identity y JWT para autenticación, con políticas para roles.
- Se documenta que contraseñas y secretos no deben almacenarse en texto plano.
- Se contempla almacenamiento privado para casos de prueba y código.
- Se contempla proteger la extracción ZIP contra rutas maliciosas y archivos excesivos.
- Judge0 debe ser consumido únicamente por el backend.
- Se propone persistencia antes de SignalR y recuperación por HTTP.
- CORS restringido y HTTPS forman parte de la configuración pendiente de los entornos desplegados.

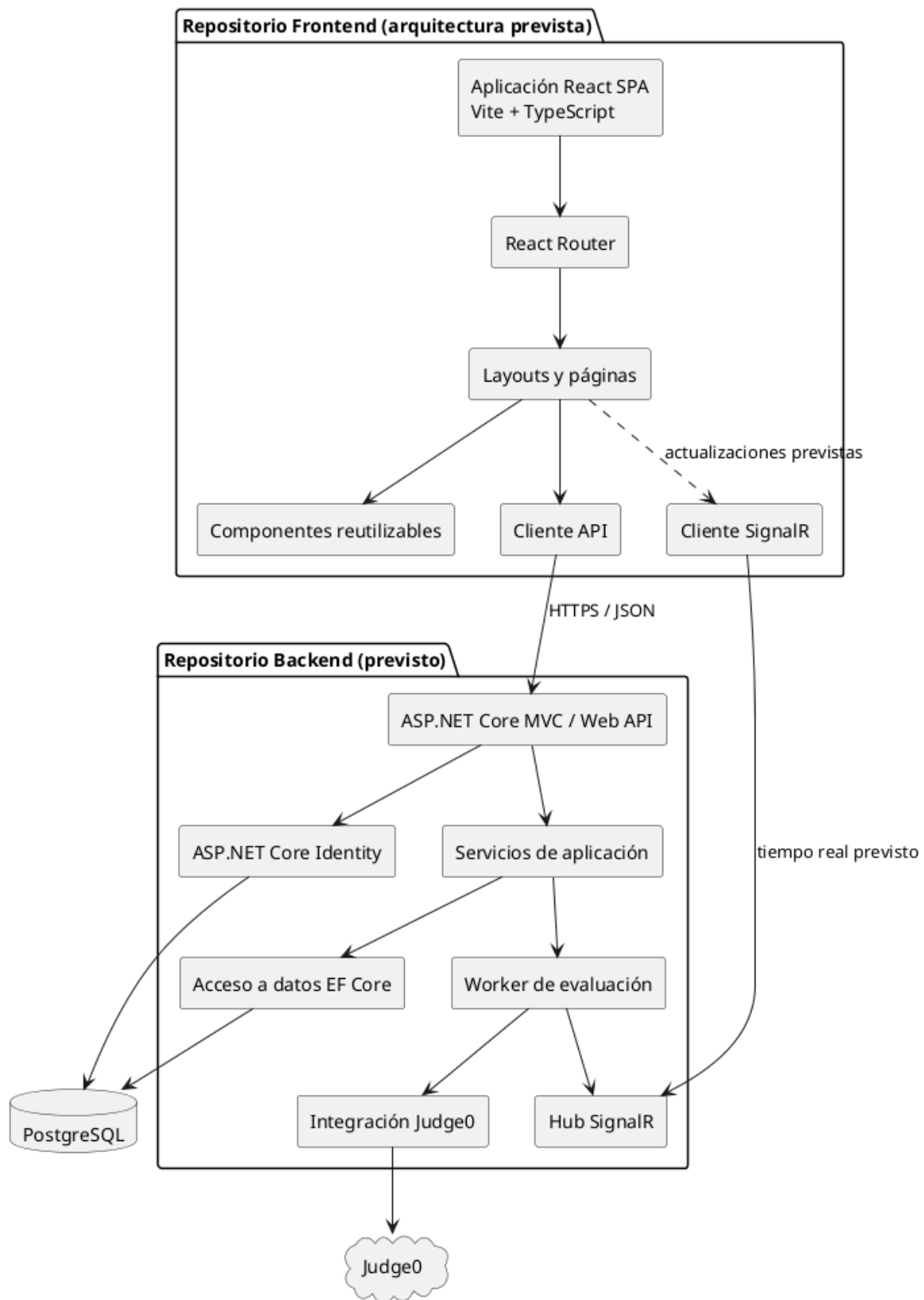


Figura 7: Arquitectura de componentes y separación de repositorios.

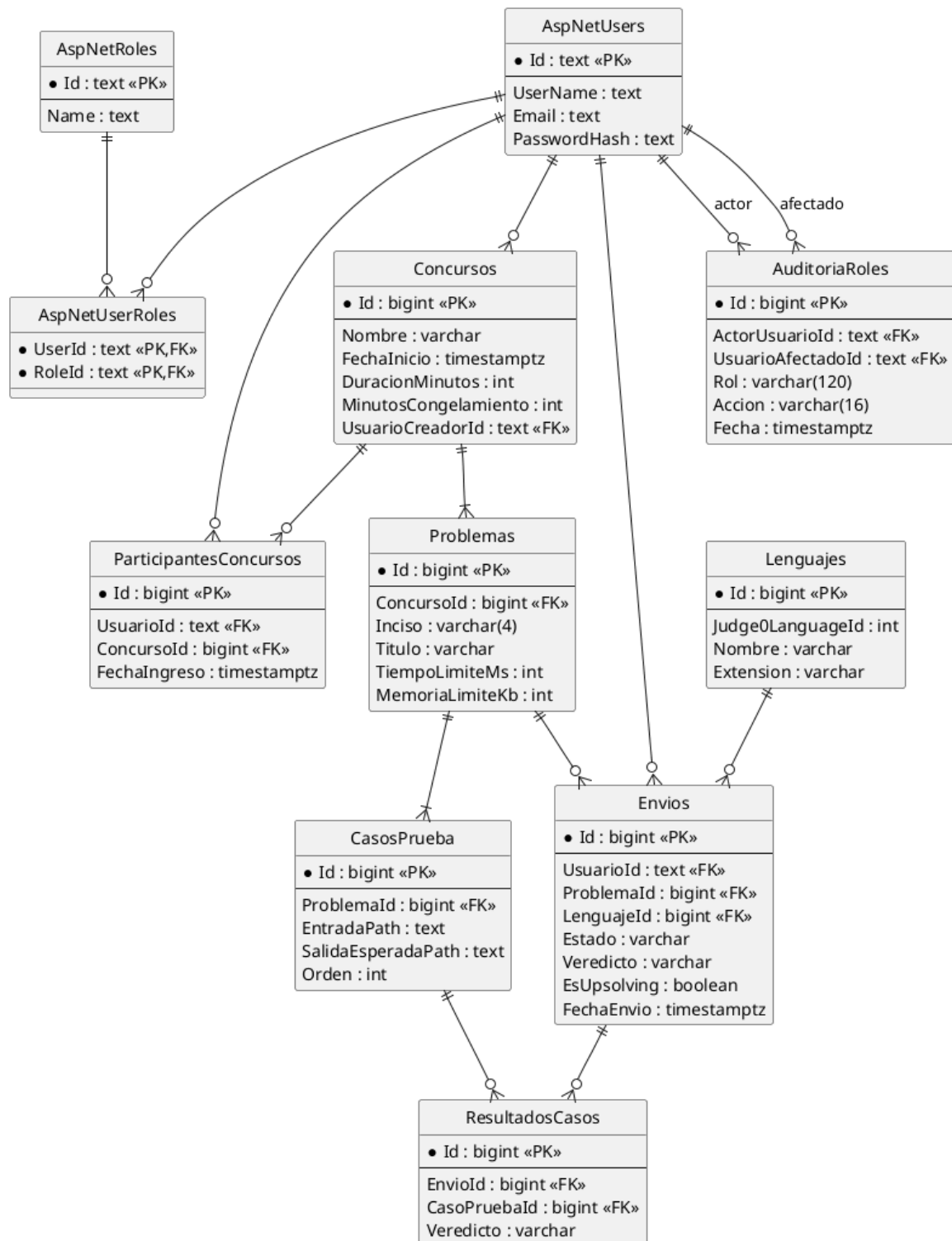


Figura 8: Modelo relacional propuesto.

12. Sprint 0 y fábrica de software

El Sprint 0 está documentado como plan para inicializar repositorios y ramas, React con Vite y TypeScript, ASP.NET Core MVC/Web API, PostgreSQL, Identity, OpenAPI, CORS y los spikes de Judge0 y SignalR. No hay evidencia en este checkout de que estas tareas, un pipeline o una prueba de integración estén configurados o ejecutados.

13. Ready to Sprint

El estado documentado es **Ready to Sprint condicionado**. Se propone iniciar la infraestructura, el diseño y las historias de identidad una vez que el frontend y el backend estén disponibles en sus repositorios. Antes de implementar la evaluación completa deben cerrarse:

- URL y estructura del repositorio backend.
- Presencia y configuración verificable del frontend React/Vite en su repositorio o en este checkout.
- Contrato OpenAPI inicial.
- Variante y disponibilidad de Judge0.
- Formato exacto del paquete ZIP.
- Almacenamiento privado de código y casos.
- Regla final de desempate del ranking.

14. Conclusiones

UPDS JUDGE posee un alcance documentado coherente para una plataforma competitiva académica, pero su complejidad principal estará en la ejecución segura, la consistencia de estados y las reglas de ranking. La separación propuesta de repositorios y ASP.NET Core MVC/Web API delimita responsabilidades. La documentación disponible define problema, MVP, backlog, criterios, UML, persistencia preliminar, seguridad propuesta y plan de fábrica, sin evidencia suficiente para afirmar funcionalidades implementadas. El enlace del backend continúa pendiente y el código del frontend React/Vite no está presente en este checkout.

A. Fuentes internas disponibles

- Wireframes exportados desde Figma.
- Modelo de datos preliminar.
- Archivo Draw.io con casos de uso.
- Captura del backlog.
- Checklist maestra de actividades académicas.

B. Archivos complementarios

El repositorio incorpora fuentes PlantUML, un script SQL inicial, imágenes y capturas existentes, y los documentos Markdown detallados. Las imágenes incluidas en este informe se refieren a archivos presentes en `docs/images/`; no se incorpora evidencia pendiente como si estuviera disponible.